

ANÁLISIS DEL RETO

Danna Jael Alape Mosquera, 202410457, d.alape@uniandes.edu.co

Nicolas Rene Sarmiento Melo, código 202524791, nr.sarmiento@uniandes.edu.co

Juan José Castro Maldonado, 202520030, j.castrom23@uniandes.edu.co

Requerimiento 1

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

En este requerimiento se busca determinar si existe una ruta entre una zona de navegacion de origen y otra de destino. Para resolverlo usamos el grafo de distancia (catalog["g_distance"]) y aplicamos BFS, ya que se pide encontrar el camino con menor numero de arcos, no con menor distancia real o tiempo. Despues de utilizar BFS desde el origen, se verifica si el destino fue alcanzado. Si existe un camino, se reconstruye usando path_to. Se muestran todos los vertices si son 10 o menos, si son mas de 10, se muestran los primeros y ultimos 5.

Entrada	Catalog, source (DEST_CLUSTER), destination (DEST_CLUSTER)
Salidas	Indica si existe o no el camino, numero total y lista de zonas en este. Para cada zona se muestra su identificador, latitud, longitud, numero de embarcaciones y los primeros y ultimos nombres de embarcaciones en la zona.
Implementado (Sí/No)	Se implemento por Juan José Castro

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Ejecutar BFS desde el origen	$O(V+E)$
Verificar si existe camino hacia el destino (Acceso)	$O(1)$
Reconstruir el camino con path_to	$O(M)$, donde M es el numero de vertices en el camino encontrado
Construir la informacion de los vertices en respuesta	$O(k)$ donde k es el numero de vertices que se tienen que mostrar
TOTAL	$O(V+E)$

Análisis

BFS garantiza el camino con el menor numero de arcos en un grafo. En este caso utilizamos `g_distance`, que demuestra la distancia por arcos y no la distancia real. La solución cumple con la instrucción porque el programa muestra todos los vertices en la trayectoria cuando son menos de 10 zonas, y muestra solo los primeros y ultimos 5 si tiene mas de 10 zonas.

Requerimiento 2

```
342 def req_2(catalog, cluster_id, radio):
343     """
344     Retorna el resultado del requerimiento 2
345     """
346     # TODO: Modificar el requerimiento 2
347
348     zona_origen = mp.get(catalog["vertices_map"], cluster_id)
349     if zona_origen is None:
350         return {"error": f"La zona '{cluster_id}' no existe en los datos."}
351
352     lat_origen = zona_origen["lat"]
353     lon_origen = zona_origen["lon"]
354
355
356     vertices_grafo = gr.vertices(catalog["g_distance"])
```

Descripción

El requerimiento busca identificar todas las zonas de navegación que se encuentran dentro de un área circular definida por una zona de interés (`cluster_id`) y un radio en kilómetros. Para cada zona encontrada se reporta su identificador, coordenadas representativas, número de registros asociados y velocidad promedio. Los resultados se presentan ordenados por distancia ascendente y, en caso de empate se presentan por el identificador del vértice.

Entrada	Catalogo, identificación del cluster y el radio en km a analizar
Salidas	Número total de zonas dentro del radio. Listado de zonas con: ID, latitud, longitud, registros asociados, velocidad promedio y distancia a la zona de origen.
Implementado (Sí/No)	Sí, Implementado por Nicolás Sarmiento

Análisis de complejidad

Se utiliza un recorrido sobre todos los vértices del grafo `g_distance` para evaluar cuáles cumplen la condición geográfica. Los vértices se obtienen directamente con `gr.vertices()`, que internamente recorre el mapa de vértices del grafo. Donde V = número de vértices del grafo y k = número de zonas encontradas dentro del radio .

Pasos	Complejidad
Buscar zona origen en vertices_map	$O(1)$
Obtener lista de vértices del grafo:	$O(V)$
Recorrer todos los vértices del grafo	$O(V)$
Por cada vértice hacer get() para obtener su info del mapa	$O(V)$
Por cada vértice: calcular haversine_distance()	$O(V)$
Por cada vértice dentro del radio usar.add_last() a la lista resultado	$O(K)$
Ordenar zonas encontradas con.merge_sort() sobre k elementos	$O(k \log k)$
TOTAL	$O(V \log V)$

Análisis

La complejidad dominante es $O(V \log V)$ debida al merge_sort final sobre las k zonas encontradas dentro del radio. En el peor caso habrá la misma cantidad de vertices que de elementos en el rango, por lo que el ordenamiento determina la complejidad total. El recorrido de los V vértices con cálculo de Haversine es $O(V)$ y no domina frente al ordenamiento.

El algoritmo es correcto porque evalúa exhaustivamente todos los vértices del grafo, garantizando que no se omite ninguna zona dentro del radio. El uso de merge_sort asegura un ordenamiento estable y eficiente, respetando el criterio de desempate por identificador de vértice especificado en el enunciado.

Requerimiento 3

```
def req_3(catalog, n):
    """
    Retorna el resultado del requerimiento 3
    """
    # TODO: Modificar el requerimiento 3
    edge_info_map = catalog["edge_info_map"]
    arcos_lista = mp.value_set(edge_info_map)

    al.merge_sort(arcos_lista, comparar_arcos_req3)

    resultado = al.new_list()
    if al.size(arcos_lista) > n:
```

Descripción

Este requerimiento identifica los corredores marítimos más transitados, es decir, los arcos que tienen el mayor número de viajes entre dos zonas de navegación. Con `edge_info_map`, el requerimiento extrae los arcos registrados y los ordena de forma descendente según la cantidad de viajes, luego de esto extrae el Top N solicitado por el usuario mediante la entrada. Al final se retorna un diccionario con la información detallada de cada conexión (origen, destino, cantidad de viajes, distancia y tiempo promedio) para facilitar su estructura en el view.

Entrada	Catálogo, n (número de conexiones a consultar)
Salidas	Listado de N conexiones más frecuentes mostrando por cada uno: zona origen, zona destino, total de viajes, distancia y tiempo promedio
Implementado (Sí/No)	Sí, por Danna Jael Alape Mosquera

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Extracción de arcos: Se llama a <code>mp.value_set(edge_info_map)</code> para obtener una lista con todos los valores almacenados en el mapa	$O(E)$, donde E equivale al número total de arcos en el mapa
Ordenamiento: Se ejecuta la función <code>merge_sort(arcos_lista, comparar_arcos_req3)</code> , con esta función se ordena la lista completa de arcos basándose en el número de viajes con el criterio definido en la función <code>comparar_arcos_req3</code>	$O(E \log E)$, debido a que la complejidad de Merge es $n \log n$
Extracción de Top N: Se utiliza un ciclo for que itera hasta el límite para construir la lista de resultado, dentro del ciclo se hacen accesos directos e inserciones al final de una lista	$O(N)$, donde N es el parámetro ingresado por el usuario para el Top
TOTAL	$O(E \log E) + N$

Análisis

La eficiencia del requerimiento se da gracias a que se aprovecha la información de `edge_info_map`. En lugar de recorrer los registros de embarcaciones desde cero, el algoritmo consulta de forma directa el mapa que ya tiene el conteo de los viajes por cada arco. Usar Merge nos da un tiempo de ejecución de $O(E \log E)$, lo cual es eficiente y escalable incluso si los datos crecen. El paso final depende de la entrada del usuario N para obtener el Top N. El uso de la tabla de hash usada al inicio evita que se realicen cálculos innecesarios.

Requerimiento 4

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Este requerimiento busca construir una red optima desde una Dest Cluster inicial hacia todo el resto de zonas alcanzables. Para resolverlo usamos Dijkstra porque se nos pide obtener los caminos de menor costo desde un punto en especifico hacia las demas zonas. Se utiliza Dijkstra sobre catalog["g_distance"], por lo que el costo corresponde a la distancia entre los clusters. Luego, se revisa el mapa visited que esta dentro de la estructura de dijkstra, se toma cada edge_from y se construye la red donde se calcula el costo total y se ordenan los arcos por origen-destino.

Entrada	Parámetros necesarios para resolver el requerimiento.
Salidas	Respuesta esperada del algoritmo.
Implementado (Sí/No)	Si, se implementó por el grupo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Dijkstra	$O((V+E)\log V)$
Recorrer el mapa de vertices visitados para identificar las zonas que se pueden alcanzar.	$O(V)$
Construir la lista de arcos con el resultado de Dijkstra	$O(V)$
Ordenar arcos por origen-destino usando merge	$O(A \log A)$ donde A es el numero de arcos en la red despues de Dijkstra
Construir el resultado	$O(1)$
TOTAL	$O((V+E) \log V + A \log A)$

Análisis

Se usa Dijkstra porque se necesita obtener la menor distancia desde un origen hacia todo el resto de clusters. Para cada vertice, dijkstra guarda la distancia minima acumulada y el vertice desde donde viene (con el menor costo) A partir de edge_from, se reconstruye la red de caminos. La red del resultado no conecta todos los vertices del grafo, solamente aquellos que son alcanzables desde otro. El costo total se calcula sumando los pesos de los arcos que forman la red.

Requerimiento 5

```
def req_5(catalog, origen, destino):  
    """  
    Retorna el resultado del requerimiento 5  
    """  
    # TODO: Modificar el requerimiento 5  
  
    if not gr.contains_vertex(catalog["g_distance"], origen):  
        return {"error": f"La zona de origen '{origen}' no existe en el grafo."}  
  
    if not gr.contains_vertex(catalog["g_distance"], destino):  
        return {"error": f"La zona de destino '{destino}' no existe en el grafo."}  
  
    estructura_dijkstra = djik.dijkstra(catalog["g_distance"], origen)
```

Descripción

El requerimiento calcula la ruta de menor costo geográfico (distancia Haversine acumulada) entre una zona de origen y una zona de destino, utilizando el algoritmo de Dijkstra sobre el grafo dirigido `g_distance`. Se reporta el costo total de la ruta, el número de zonas y arcos que la componen, y la información detallada de los vértices que la conforman (mostrando los cinco primeros y cinco últimos si la ruta supera los diez vértices).

Entrada	Catalog, id de origen e id de destino
Salidas	Indicación de si existe ruta entre las zonas. Costo total de la ruta en km. Número total de zonas y arcos en la ruta. Listado de hasta 10 vértices (5 primeros y 5 últimos) con: ID, latitud, longitud, número de embarcaciones y peso del arco hacia el siguiente vértice.
Implementado (Sí/No)	Sí, se implementó por el grupo.

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Verificar existencia de origen y destino	$O(1)$
Inicializar estructura Dijkstra	$O(1)$
Inicializar distancias de todos los vértices a infinito	$O(V)$
Ejecutar Dijkstra: cada vértice se extrae una vez de la cola de prioridad y sus arcos se relajan	$O((V + E) \log V)$
Verificar si existe ruta hacia el destino	$O(1)$
Reconstruir camino con <code>path_to</code> siguiendo <code>edge_from</code>	$O(p)$ (donde p es número de vértices que tiene el camino encontrado entre el origen y el destino)

Vaciar el stack para construir lista_ruta	$O(P)$
Construir los vértices a mostrar (máximo 10)	$O(1)$
TOTAL	$O((V + E) \log V)$

Análisis

La eficiencia de este requerimiento se basa en el uso del algoritmo de Dijkstra con cola de prioridad mínima sobre el grafo dirigido `g_distance`. Se eligió Dijkstra porque los pesos de los arcos corresponden a distancias Haversine, que siempre son valores no negativos, condición necesaria para que el algoritmo funcione correctamente. Una alternativa como BFS solo sería válida si se buscara el camino con menor número de arcos, no el de menor distancia acumulada.

Requerimiento 6

```
def req_6(catalog):
    """
    Retorna el resultado del requerimiento 6
    """
    # TODO: Modificar el requerimiento 6
    grafo_dirigido = catalog["g_distance"]
    vertices_lista = gr.vertices(grafo_dirigido)
    total_vertices = gr.order(grafo_dirigido)

    if total_vertices == 0:
        return al.new_list()
```

Descripción

Este requerimiento busca descubrir las rutas fuertemente conectadas, evaluando subredes donde se garantiza que las embarcaciones puedan ir y volver entre zonas. El algoritmo primero construye un grafo no dirigido a partir del grafo dirigido original, agregando un arco únicamente si existe una conexión en ambos sentidos, luego utiliza un recorrido bfs sobre los vértices para identificar las rutas conectadas. Finalmente, se procesa cada subred ordenando los nodos internos y ordenando la lista de subredes para retornar el top 5 de las subredes más grandes.

Entrada	Catálogo
Salidas	Número total de subredes independientes y una lista con las 5 subredes más grandes mostrando: ID de subred, total de zonas, lista de ID de zonas ordenada, total de viajes y velocidad promedio
Implementado (Sí/No)	Sí, se implementó por el grupo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Construcción del grafo no dirigido: Se llama a <code>crear_grafo_no_dirigido()</code> , la cual itera sobre los	$O(V+E)$

vértices y sus arcos adyacentes para verificar si existe el arco de regreso	
BFS: Un ciclo for recorre los vértices, si un vértice no se ha visitado se usa <code>bfs_vertex()</code> . BFS revisa cada vértice y cada arco del nuevo grafo una vez	$O(V+E)$, ya que recorre el grafo creado
Procesamiento de subredes: Por cada subred descubierta, se llama a <code>procesar_subred()</code> , aquí se calculan promedios y se ejecuta <code>merge_sort()</code> para ordenar los k vértices de la subred	$O(V \log V)$, en el peor caso la sumatoria de los ordenamientos de cada subred k es igual a V y no excede el costo de ordenar todos los V
Ordenamiento de subredes: Se ejecuta un <code>merge_sort(subarcos_list, comparar_subredes)</code> para ordenar las C subredes encontradas por su tamaño	$O(V \log V)$, en el peor caso C es igual a V
TOTAL	$O(V \log V + E)$

Análisis

Para este requerimiento se construyó un grafo no dirigido en el requerimiento ya que ayuda a una búsqueda más fácil de las subredes válidas, al filtrar los arcos desde el principio, el problema se reduce a buscar los componentes conectados, por esta razón usamos BFS que resuelve esto de manera óptima en $O(V+E)$.

El uso de `visited_map`, garantiza que ningún vértice se procese más de una vez durante su búsqueda, aumentando la eficiencia. Los algoritmos de ordenamiento implementados en los pasos finales tienen un costo de $O(V \log V)$, lo cual asegura un rendimiento veloz cumpliendo con los criterios del requerimiento.